

BANCO DIGITAL

Plano de Execução do Projeto Mobile - Flutter

Documento de organização para o grupo

Gestor Técnico do Projeto	Kauã
Frontend principal	Maria
Frontend avançado	Neto
UI, componentes e documentação	Jean
Plugins, comprovante e testes	Rian

Foco: entregar primeiro o MVP obrigatório e depois refinar o visual.

Prazo interno: finalizar funcionalidades até o Dia 4. Dia 5 apenas para testes, correções e APK.

1. Objetivo do projeto

Criar um aplicativo mobile de Banco Digital usando Flutter, com login, painel principal, cotação de moedas em tempo real, transferência entre contas, banco de dados conectado, plugin extra e APK final para Android.

O projeto será dividido em duas partes principais:

- Mobile: aplicativo Flutter feito em Dart.
- Backend/API: servidor Node.js com Express feito em JavaScript.
- Banco de dados: Firebase Firestore.

2. Decisão sobre repositório

A recomendação oficial para este projeto é usar monorepo: um único repositório no GitHub com o app Flutter e o backend separados em pastas.

```
banco-digital/  
  mobile/    -> projeto Flutter  
  backend/   -> API Node.js + Express  
  docs/      -> documentação, prints, checklist e anotações  
  README.md
```

Isso não tira a profissionalidade. O que torna o projeto profissional é a organização, a arquitetura clara, commits bem escritos, separação de responsabilidades e integração controlada.

O monorepo é indicado porque o prazo é curto, o app e o backend dependem um do outro, e o Gestor Técnico consegue revisar tudo em um único lugar antes da entrega.

3. Obrigações oficiais do projeto para entrega

Critério	Pontos	Como vamos cumprir
Botões, inputs e widgets básicos	1,0 pt	Todas as telas terão botões funcionais, campos de texto, cards, navegação e componentes reutilizáveis.
API com conexão	1,0 pt	O app consumirá API de cotação e também se comunicará com o backend Node.js.
Telas obrigatórias	1,0 pt	Login, Principal, Cotação e Transferência são prioridade máxima.
Plugin extra	1,0 pt	Usar share_plus para compartilhar comprovante. Se der tempo, local_auth para biometria.
Banco de dados	1,0 pt	Firebase Firestore para usuários, contas, saldos e transações.
Criatividade	0,5 pt	Tema escuro, visual roxo/neon, Pix, comprovante, scanner, perfil e componentes visuais.
APK otimizado	0,5 pt	Gerar APK com flutter build apk --split-per-abi.

Entrega obrigatória: o projeto deve ser entregue zipado em formato .zip, contendo código-fonte completo e APK split na pasta build/app/outputs. Alternativamente, pode ser enviado link do FlutLab, mas a preferência do grupo é entregar ZIP com código + APK.

4. Prioridade estratégica do grupo

A prioridade é garantir os pontos obrigatórios antes de expandir funcionalidades. A ordem de entrega deve ser:

1. Login funcionando.
2. Tela Principal/Home funcionando.
3. Cotação funcionando com API real.
4. Transferência funcionando.
5. Banco de dados salvando e buscando informações.
6. Plugin funcionando.
7. APK split gerado.
8. Depois: scanner, perfil, idioma, configurações, animações e refinamento visual.

Regra principal: se uma tela bonita não ajuda diretamente na nota, ela só entra depois das telas obrigatórias funcionarem.

5. MVP obrigatório

- Login com e-mail e senha.
- Home com saldo principal.
- Cotação de dólar/euro consumindo API externa.
- Transferência com valor e destinatário.
- Confirmação de transferência.
- Histórico básico de transações.
- Compartilhamento de comprovante.
- Banco Firestore conectado.
- APK split gerado.

Funcionalidades extras somente se sobrar tempo

- QR scanner.
- Meu QR Code.
- Tela de idioma.
- Tela completa de perfil.
- Alterar senha.
- Biometria.
- Notificações.
- Dark mode configurável.
- Animações avançadas.

6. Tecnologias e ferramentas oficiais

Área	Ferramentas
Frontend mobile	Flutter, Dart, Provider, HTTP, Shared Preferences, Flutter Secure Storage, Local Auth, Share Plus, Intl
Backend	Node.js, Express, JavaScript, Firebase Admin SDK, Bcryptjs, JWT, Dotenv, Cors, Axios, Zod
Banco de dados	Firebase Firestore
API de cotação	AwesomeAPI / Economia AwesomeAPI
Ferramentas	VS Code, Android Studio, Git/GitHub, Postman ou Insomnia, Flutter Doctor, Java JDK 17+

Observação importante: Firebase não é linguagem. Firebase é uma plataforma/banco de dados. As linguagens do projeto serão Dart no Flutter e JavaScript no backend.

7. Arquitetura escolhida

A arquitetura será uma Clean Architecture simplificada, usando ideias de MVC e Repository Pattern. Como o projeto é pequeno e o prazo é curto, não será usada uma arquitetura exagerada.

Fluxo no Flutter

Tela -> Controller/Provider -> Service -> Backend/API -> Banco de Dados

A tela não deve chamar HTTP diretamente. Ela chama o controller/provider, e o controller chama o service.

Fluxo no backend

Route -> Controller -> Service -> Repository -> Firestore

Camada	Responsabilidade
Routes	Definem os endpoints da API.
Controllers	Recebem a requisição e devolvem a resposta.
Services	Guardam regras de negócio, como validar saldo e criar transação.
Repositories	Acessam o banco de dados Firestore.
Middlewares	Cuidam de autenticação, validação e tratamento de erros.

8. Estrutura de pastas

Estrutura do Flutter

```
mobile/  
  lib/  
    main.dart  
    app.dart  
  core/  
    routes/app_routes.dart  
    theme/app_theme.dart  
    constants/api_constants.dart  
    utils/validators.dart  
    utils/currency_formatter.dart  
  models/  
    user_model.dart  
    account_model.dart  
    transaction_model.dart  
    exchange_rate_model.dart  
  services/  
    api_client.dart  
    auth_service.dart  
    account_service.dart  
    transfer_service.dart  
    quote_service.dart  
    biometric_service.dart  
    storage_service.dart  
    share_service.dart  
  controllers/  
    auth_controller.dart  
    dashboard_controller.dart  
    transfer_controller.dart  
    quote_controller.dart  
    profile_controller.dart  
    settings_controller.dart  
    qr_controller.dart  
  screens/  
    auth/  
    home/  
    pix/  
    quotes/  
    settings/  
    profile/  
  widgets/  
    buttons/  
    cards/  
    bottom_navigation/  
    inputs/  
    modals/  
    loading/  
    feedback/
```

Estrutura do backend

```
backend/  
  src/  
    server.js  
    app.js  
  config/  
    firebase.js  
    env.js  
  routes/  
    auth.routes.js  
    account.routes.js  
    transfer.routes.js  
    quote.routes.js  
  controllers/  
    auth.controller.js  
    account.controller.js  
    transfer.controller.js  
    quote.controller.js  
  services/  
    auth.service.js  
    account.service.js  
    transfer.service.js  
    quote.service.js  
  repositories/  
    user.repository.js  
    account.repository.js  
    transaction.repository.js  
  middlewares/  
    auth.middleware.js  
    error.middleware.js  
    validate.middleware.js  
  utils/  
    generateToken.js  
    validators.js  
    formatMoney.js
```

9. Rotas e endpoints principais

Endpoint	Função
POST /auth/register	Cadastrar usuário inicial para teste.
POST /auth/login	Autenticar usuário e retornar token.
GET /auth/me	Buscar dados do usuário logado.
GET /account/balance	Buscar saldo.
GET /account/summary	Buscar resumo da conta.
GET /quotes	Buscar cotação atual das moedas.
POST /transfers	Criar transferência.
GET /transfers/history	Buscar histórico de transações.

10. Escopo visual real do aplicativo

Com base nas telas enviadas pelo grupo de frontend, o aplicativo tem um escopo visual maior do que o mínimo exigido. Isso será tratado como diferencial, mas não pode atrasar o MVP obrigatório.

- Área Pix completa.
- Tela de transferência.
- Tela de comprovante.
- Dashboard principal.

- Cotação de moedas.
- Perfil.
- Configurações.
- Alterar senha.
- Idioma.
- Scanner QR Code.
- Meu QR Code.
- Histórico/transações.
- Tela de pagamentos.
- Navegação inferior personalizada.
- Componentes reutilizáveis.

Estratégia para o frontend: primeiro estrutura e navegação; depois componentes reutilizáveis; depois integração com APIs; por último, refinamento visual.

11. Componentes reutilizáveis obrigatórios

- Botão principal.
- Campo de texto padrão.
- Campo de senha.
- Card de saldo.
- Card de moeda.
- Card de transação.
- Bottom navigation.
- Header padrão.
- Loading spinner.
- Toast/snackbar.
- Mensagem de erro.

12. Divisão de responsabilidades

Kauã - Gestor Técnico do Projeto + Backend + Revisão final

- Criar e organizar o repositório no GitHub.
- Criar a estrutura do backend em Node.js + Express.
- Configurar Firebase Firestore no backend.
- Criar rotas de autenticação, conta, cotação e transferência.
- Implementar hash de senha com bcryptjs.
- Implementar autenticação com JWT.
- Revisar merges e pull requests.
- Integrar backend com frontend junto com Maria e Neto.
- Fazer checklist final do projeto.
- Orientar a geração do APK.
- Garantir que todos entreguem suas partes até o Dia 4.

Maria - Frontend principal: Login + Home

- Criar tela de login.
- Criar validação visual de e-mail e senha.
- Conectar login com backend.
- Salvar sessão simples com Shared Preferences/Secure Storage.
- Criar tela principal/home.
- Criar cards de saldo e resumo da conta.
- Consumir dados da conta vindos do backend.
- Ajudar Jean com componentes visuais.

Neto - Frontend avançado: Cotação + Transferência + Estado

- Criar tela de cotação.

- Consumir endpoint de cotação.
- Exibir dólar, euro e outras moedas.
- Criar tela de transferência.
- Implementar passagem de argumentos por rota.
- Validar valor de transferência no front.
- Integrar transferência com backend.
- Configurar Provider/controllers principais.
- Ajudar Maria na integração geral do Flutter.

Jean - UI, componentes e documentação visual

- Criar componentes reutilizáveis simples.
- Ajustar cores, fontes, espaçamentos e ícones.
- Criar layout inicial das telas com base no design.
- Fazer prints das telas funcionando.
- Ajudar no README com instruções de execução.
- Testar se os botões levam para as telas corretas.

Rian - Plugin extra + testes manuais + comprovante

- Implementar ou ajudar a implementar share_plus.
- Criar tela simples de comprovante após transferência.
- Adicionar botão Compartilhar comprovante.
- Ajudar no teste manual das telas.
- Conferir se o app abre no emulador/celular.
- Validar login, home, cotação e transferência.
- Montar checklist de bugs encontrados.

13. Backlog do produto

Prioridade Alta - precisa existir para entregar

ID	Funcionalidade	Responsável	Apoio
B01	Criar repositório e estrutura base	Kauã	Todos
B02	Criar projeto Flutter	Maria/Neto	Kauã
B03	Criar backend Express	Kauã	Neto
B04	Configurar Firebase Firestore	Kauã	-
B05	Criar tela de login	Maria	Jean
B06	Criar autenticação no backend	Kauã	Maria
B07	Criar dashboard/home com saldo	Maria	Jean
B08	Criar tela de cotação	Neto	Jean
B09	Integrar AwesomeAPI	Neto/Kauã	-
B10	Criar tela de transferência	Neto	Rian
B11	Criar regra de saldo suficiente	Kauã/Neto	-
B12	Salvar histórico de transações	Kauã	Neto
B13	Usar rotas nomeadas	Neto	Maria
B14	Usar argumentos entre telas	Neto	-
B15	Plugin share_plus	Rian	Neto
B16	Gerar APK final	Kauã	Todos

Prioridade Média - melhora a nota

ID	Funcionalidade	Responsável	Apoio
B17	Dark mode simples	Jean	Maria
B18	Animações simples	Jean	Rian
B19	Biometria com local_auth	Maria/Rian	Kauã
B20	Tela de comprovante bonita	Rian	Jean
B21	Loading e mensagens de erro	Maria/Neto	Jean

14. Sprints de 5 dias

A meta é terminar tudo até o Dia 4 e deixar o Dia 5 apenas para testes, correções e empacotamento.

Sprint	Objetivo
Dia 1 - Fundação	Criar repositório, backend base, Flutter base, rotas iniciais, componentes principais e rota /health.
Dia 2 - Login e Home	Autenticação, Firestore, login integrado, token salvo e home mostrando saldo.
Dia 3 - Cotação e Transferência	Cotação com API, transferência com validação, saldo atualizado e histórico salvo.
Dia 4 - Integração final	Revisar tudo, corrigir bugs, juntar branches, congelar features e preparar build.
Dia 5 - Testes e entrega	Testar fluxos, gerar APK split, fechar ZIP e revisar documentação final.

Sprint 1 - Dia 1: Fundação do projeto

Pessoa	Tarefas
Kauã	Criar repositório, backend, instalar dependências, configurar Firebase e rota GET /health.
Maria	Criar projeto Flutter, instalar pacotes e iniciar tela de login.
Neto	Configurar rotas nomeadas e telas vazias: home, cotação e transferência.
Jean	Criar paleta inicial, custom_button e custom_text_field.
Rian	Criar tela de comprovante, testar share_plus e criar checklist de testes.

Entregável do Dia 1: projeto abre, navega entre telas e backend responde /health.

Sprint 2 - Dia 2: Login, banco e home

Pessoa	Tarefas
Kauã	Criar collections users, accounts e transactions; rotas de register/login; JWT; middleware; account summary.
Maria	Integrar login, salvar token, redirecionar para home, mostrar loading/erro e saldo da API.
Neto	Criar api_client, auth_service, account_service, auth_controller e home_controller.
Jean	Melhorar visual da home, balance_card, transaction_card e espaçamentos.
Rian	Testar login, documentar erros e iniciar botão de compartilhar comprovante.

Entregável do Dia 2: login funciona, token é salvo e home mostra saldo vindo do backend.

Sprint 3 - Dia 3: Cotação e transferência

Pessoa	Tarefas
Kauã	Criar /quotes, /transfers e /transfers/history; validar saldo; atualizar saldo; salvar transação.
Maria	Ajustar home, histórico/resumo e mensagens de erro/sucesso.
Neto	Finalizar cotação, quote_service, quote_controller, transferência, transfer_service e argumentos de rota.
Jean	Melhorar visual da cotação e transferência, estados vazios e mensagens amigáveis.
Rian	Finalizar comprovante e fluxo compartilhar com share_plus.

Entregável do Dia 3: cotação funciona, transferência funciona, saldo atualiza e histórico é salvo.

Sprint 4 - Dia 4: Integração final e congelamento de features

Pessoa	Tarefas
Kauã	Revisar backend, corrigir integração, revisar PRs, juntar na main e preparar build.
Maria	Revisar login/home, responsividade e testes em emulador/celular.

Neto	Revisar cotação/transferência, argumentos de rota e Provider/controllers.
Jean	Revisão visual final, README e prints.
Rian	Testar todos os fluxos, confirmar plugin e registrar bugs críticos.

Entregável do Dia 4: app completo, integrado e pronto para testes finais. Nenhuma feature nova entra depois disso.

Sprint 5 - Dia 5: Testes, correções e entrega

- Testar login.
- Testar navegação.
- Testar cotação.
- Testar transferência.
- Testar saldo após transferência.
- Testar histórico.
- Testar compartilhamento de comprovante.
- Testar APK instalado no Android.
- Fechar ZIP final com código, APK, README e nomes dos integrantes.

Entregável do Dia 5: ZIP final pronto para entrega.

15. Regras de Git do grupo

```
main          -> versão estável
backend-kaia  -> backend
frontend-maria -> login e home
frontend-neto  -> cotação e transferência
ui-jean       -> componentes e design
plugin-rian   -> share_plus, comprovante e testes
```

- Ninguém envia direto para main sem revisão do Gestor Técnico.
- Cada pessoa trabalha na sua branch.
- Fazer commit com mensagem clara.
- Sempre avisar no grupo quando terminar uma tarefa.
- Antes de mexer, dar git pull.
- Antes de entregar tarefa, testar se o app roda.

16. Definition of Done

- Código funcionando.
- Não quebrar outras telas.
- Estar na pasta correta.
- Ter nome claro de arquivo, classe e função.
- Tela ou rota testada.
- Responsável avisou no grupo o que foi feito.
- Kauã revisou antes de ir para main.

17. Checklist final obrigatório

- [] Login com e-mail e senha funcionando.
- [] Home mostrando saldo e resumo da conta.
- [] Tela de cotação mostrando moedas atualizadas.
- [] Tela de transferência funcionando.
- [] Saldo validado antes da transferência.
- [] Saldo atualizado após transferência.
- [] Histórico de transações salvo.
- [] Rotas nomeadas funcionando.
- [] Pelo menos uma rota com argumentos funcionando.
- [] Plugin extra funcionando.

- [] Banco de dados conectado.
- [] APK gerado.
- [] Código organizado em pastas.
- [] README atualizado.
- [] Nenhum erro crítico no app.

18. Comandos úteis

Flutter

```
flutter doctor
flutter pub get
flutter run
flutter build apk --split-per-abi
```

Pacotes Flutter

```
flutter pub add http
flutter pub add provider
flutter pub add shared_preferences
flutter pub add flutter_secure_storage
flutter pub add local_auth
flutter pub add share_plus
flutter pub add intl
```

Backend

```
npm init -y
npm install express cors dotenv firebase-admin bcryptjs jsonwebtoken zod axios helmet morgan
npm install nodemon --save-dev
npm run dev
```

19. Mensagem pronta para mandar no grupo

Pessoal, organizei o plano do projeto para a gente não se perder e conseguir entregar no prazo.

A divisão ficou assim:

- Kauã: Gestor Técnico, backend, banco, API e revisão final.
- Maria: login, home e integração principal do app.
- Neto: cotação, transferência, rotas e estado.
- Jean: componentes visuais, layout, README e prints.
- Rian: plugin de compartilhamento, comprovante e testes manuais.

A meta é terminar tudo até o Dia 4. No Dia 5, a gente só testa, corrige bugs críticos, gera o APK e fecha o ZIP final.

Cada pessoa vai trabalhar na sua branch e nada vai direto para a main sem revisão. Vamos começar pela Sprint 1: estrutura do projeto, backend base, Flutter base, rotas iniciais e componentes principais.